

SISTEMAS DISTRIBUÍDOS • AULA 23

Sistemas Tolerantes a Falhas III

Recuperação e Sistemas Resilientes

Terceira e última aula sobre tolerância a falhas em sistemas distribuídos

O que vamos ver hoje

01 Do modelo de falha à recuperação

Retomando brevemente detecção (Aula 21) e replicação (Aula 22): o que falta quando uma falha já aconteceu.

02 Recuperação para trás

Backward recovery: voltar a um estado consistente anterior usando pontos de verificação (checkpoints).

03 Recuperação para frente

Forward recovery: reconstruir o estado correto a partir de informação redundante, sem retroceder.

04 Checkpointing e logging de mensagens

Como salvar estado periodicamente e complementar com registro de mensagens trocadas.

05 Estudo de caso: Netflix e Chaos Engineering

Projetar para a falha: testar resiliência injetando falhas de propósito em produção.

Da detecção e da replicação até a recuperação

Nas duas últimas aulas vimos como perceber que algo falhou (modelos de falha, timeouts, detectores) e como mascarar a falha mantendo réplicas ativas. Falta a terceira peça: o que acontece quando um processo volta a funcionar, ou quando o sistema precisa restabelecer um estado correto depois do erro.

Aula 21 — Detecção

Perceber que um processo ou canal falhou (timeouts, heartbeats, detectores não confiáveis).

Aula 22 — Replicação

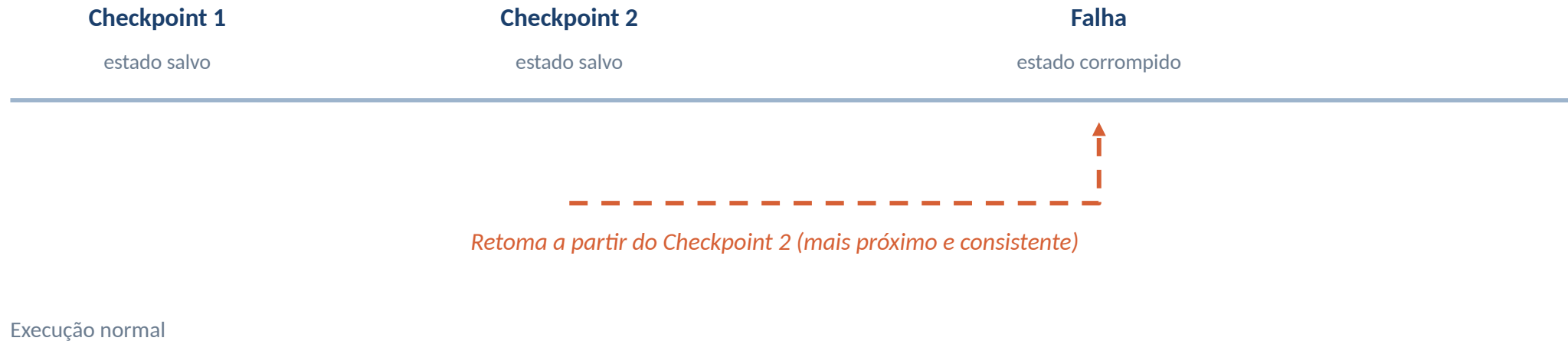
Manter cópias redundantes para que a falha de uma réplica não interrompa o serviço.

Aula 23 — Recuperação

Restabelecer um estado correto depois da falha — para trás ou para frente.

Backward recovery: voltar a um estado anterior

A ideia central: periodicamente, o sistema salva seu estado em um ponto de verificação (checkpoint). Se ocorre uma falha, em vez de tentar consertar o estado corrompido, o processo simplesmente retorna ao último checkpoint consistente e recomeça a execução a partir dali.



Custo do rollback: quanto mais recente o checkpoint, menos trabalho se perde — mas checkpoints frequentes custam mais overhead de execução (E/S, pausa do processo, espaço de armazenamento). É um equilíbrio de projeto.

Checkpointing: salvar estado para permitir retomada

O que é um checkpoint

É um ponto de verificação: uma cópia do estado de um processo (memória, variáveis, posição de execução) salva em armazenamento estável, em um instante em que o processo está consistente. Ao falhar, o processo é reiniciado a partir do último checkpoint em vez de do zero.

Checkpoint individual x checkpoint coordenado

Em um sistema distribuído, processos que se comunicam não podem salvar estados independentes sem cuidado: se um processo volta a um checkpoint antigo, mas outro já processou uma mensagem baseada no estado 'futuro' dele, o conjunto de checkpoints fica inconsistente (efeito dominó).

CONEXÃO COM A AULA 19

Checkpoint coordenado = snapshot distribuído

O algoritmo de Chandy-Lamport, visto na Aula 19, resolve exatamente o problema de obter um conjunto consistente de estados locais mais o conteúdo dos canais — um corte consistente, sem efeito dominó.

Usar marcadores para capturar, junto com o estado de cada processo, as mensagens em trânsito nos canais é o que garante que o conjunto de checkpoints sirva como ponto de retomada válido para todo o sistema — não só para um processo isolado.

Forward recovery: reconstruir sem retroceder

Em vez de voltar a um estado passado, a recuperação para frente usa informação redundante já disponível no momento da falha para calcular diretamente um estado correto e seguir em frente — sem repetir trabalho já feito.

Quando funciona bem

O tipo de erro é conhecido e limitado (ex.: perda de um pacote, um bit corrompido).

Existe redundância suficiente para reconstruir o dado correto (réplicas, códigos de correção de erro, paridade).

Exemplos típicos

Retransmissão seletiva de um pacote perdido usando ACKs — não recomeça a conexão inteira.

Reconstrução de um bloco de disco perdido a partir da paridade em um arranjo RAID.

Limite importante: recuperação para frente só é viável quando se consegue prever, de antemão, os tipos de erro possíveis — não serve para falhas arbitrárias ou não previstas, onde o backward recovery continua sendo a alternativa geral.

Logging de mensagens

Checkpoints sozinhos podem perder mensagens processadas entre o último checkpoint e a falha. O logging de mensagens registra, em armazenamento estável, as mensagens recebidas (e/ou enviadas) por um processo — permitindo reconstruir exatamente o processamento perdido ao reiniciar do checkpoint.

Checkpoint salvo

Estado do processo é gravado em um instante consistente.

Mensagens logadas

Cada mensagem recebida depois do checkpoint é registrada em log estável, antes de ser processada.

Falha ocorre

O processo perde seu estado em memória; o checkpoint e o log sobrevivem em armazenamento estável.

Replay do log

O processo reinicia do checkpoint e reprocessa as mensagens logadas, na ordem, até alcançar o estado perdido.

Assumir que tudo vai falhar

"Design for failure": em sistemas distribuídos de larga escala, falhas de hardware, rede e software não são exceção — são estatisticamente certas de acontecer com frequência. O projeto do sistema precisa partir dessa premissa, em vez de tratá-la como caso raro.

1

Redundância por padrão

Nenhum componente único deve ser ponto único de falha (SPOF).

2

Detectar rápido

Falhas silenciosas custam mais caro que falhas visíveis e sinalizadas cedo.

3

Recuperação automática

O sistema se recupera sem intervenção manual sempre que possível.

4

Testar a falha, não só o sucesso

Validar resiliência provocando falhas de propósito, e não apenas esperando que ocorram.

Netflix e o Chaos Monkey

A Netflix opera um dos maiores sistemas distribuídos do mundo sobre infraestrutura de nuvem, onde a falha de máquinas individuais é rotina. Em vez de só se preparar para a falha, a empresa criou o Chaos Engineering: a prática de injetar falhas deliberadamente em produção para verificar, de forma controlada, se o sistema realmente é resiliente como o projeto supõe.



Chaos Monkey

Ferramenta que desliga instâncias de servidor aleatoriamente em produção durante o horário comercial, forçando as equipes a projetar serviços que toleram essa perda sem impacto ao usuário.



Simian Army

Família de ferramentas que estende a ideia: simula latência de rede, indisponibilidade de zonas inteiras e outras falhas realistas, não só a queda de uma máquina.



Por que funciona

Falhas testadas de forma controlada, em horário monitorado, revelam pontos fracos antes que uma falha real e não planejada cause um incidente grave.

O que levar desta aula

Backward recovery

Volta a um checkpoint (ponto de verificação) consistente anterior e recomeça a partir dali.

Forward recovery

Usa redundância disponível para reconstruir o estado correto sem retroceder — só viável para erros previsíveis.

Checkpointing + logging

Checkpoints coordenados evitam efeito dominó (conexão com Chandy-Lamport, Aula 19); logs de mensagem cobrem o intervalo entre checkpoints.

Design for failure

Netflix e o Chaos Monkey mostram: testar a resiliência de propósito, não só esperar a falha acontecer.

PRÓXIMA AULA

Aula 24 — Sistemas Operacionais Distribuídos

Conceitos e Arquitetura

Com a sequência de tolerância a falhas fechada — modelos e detecção, replicação e redundância, recuperação e resiliência — a próxima etapa muda de foco: como o sistema operacional em si é estruturado para suportar computação distribuída.

Fim da 3ª e última aula da sequência sobre tolerância a falhas.

Bibliografia desta aula

TANENBAUM, Andrew S.; VAN STEEN, Maarten.

Distributed Systems: Principles and Paradigms (ou edição atual Distributed Systems). Pearson.

Capítulo sobre tolerância a falhas: recuperação para trás e para frente, checkpointing, logging de mensagens.

COULOURIS, George et al.

Sistemas Distribuídos: Conceitos e Projeto. 5ª ed. Porto Alegre: Bookman.

Capítulo sobre tolerância a falhas e recuperação: pontos de verificação e reconstrução de estado após falha.

BASIRI, Ali et al.

"Chaos Engineering". IEEE Software, v. 33, n. 3, 2016.

Artigo da equipe de resiliência da Netflix descrevendo a prática de Chaos Engineering e o Chaos Monkey.